

CS5300 Parallel and Concurrent Programming

Course Project : Parallel Algorithms for Sorting

Anakala Sashikiran, Anup Chavan and Shravan S

April 2026

1 Introduction

Sorting algorithms are fundamental to Computer Science and finds numerous applications in real life due to its applications in ordering data.

We compare the implementations of the following parallel sorting algorithms

- **Cole's Parallel Merge Sort** [1]

Cole gives a parallel implementation of merge sort on a computation model where no two processors are allowed to simultaneously read from or write to the same shared memory. It uses n processors and $O(\log n)$ time.

This was made possible due to the utilisation of pipelining technique applied to the merge tree: rather than waiting for each level of merging to complete before starting the next, Cole's algorithm overlaps the merge operations across consecutive levels.

At every step each node in the merge tree maintains a sample of its partially merged result, and these samples are propagated upward so that coarser-level merges can begin before finer-level merges have finished.

- **Powers' Parallelized QuickSort** [2]

Powers described a parallelized quicksort that can operate in $O(\log n)$ time on a similar computation model with n processors by performing partitioning implicitly via a parallel prefix sum.

In each recursive level the algorithm proceeds as follows:

1. A pivot is selected (e.g., median or random element).
2. Every processor compares its element with the pivot and writes a 0 or 1 flag.
3. A parallel prefix sum over these flags computes the destination index for each element.
4. Elements are moved to their computed positions, completing the partition in one parallel step.

- **Odd–Even Transposition Sort** [3, 4]

The odd–even transposition sort was originally presented by Habermann. Baudet and Stevenson later extended this to an optimal merge-splitting variant for parallel computers.

Processors are arranged in a linear array, and the algorithm alternates between two phases over n rounds:

- **Odd phase:** Processors at positions (1, 2), (3, 4), (5, 6), ... compare and exchange their elements.
- **Even phase:** Processors at positions (0, 1), (2, 3), (4, 5), ... compare and exchange their elements.

2 Implementation details

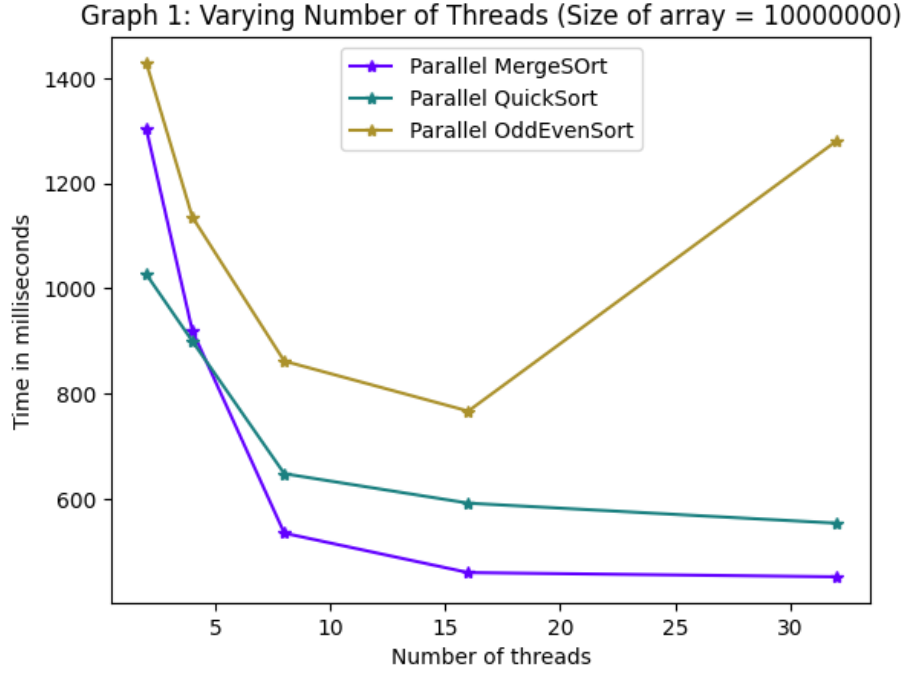
- We have implemented the algorithms and the logic to run the experiment in cpp using cpp threads and data structures given in standard library.
- The following are taken as command line arguments : number of elements (N), number of threads and a flag indicating whether or not to generate logs.
- Then a random vector with N elements is generated.
- After that, we call our implementations of the following three algorithms with this random vector five times and get the time taken averaged over five observations to reduce erratic changes.
- Finally the observations are plotted using matplotlib

3 Experiments

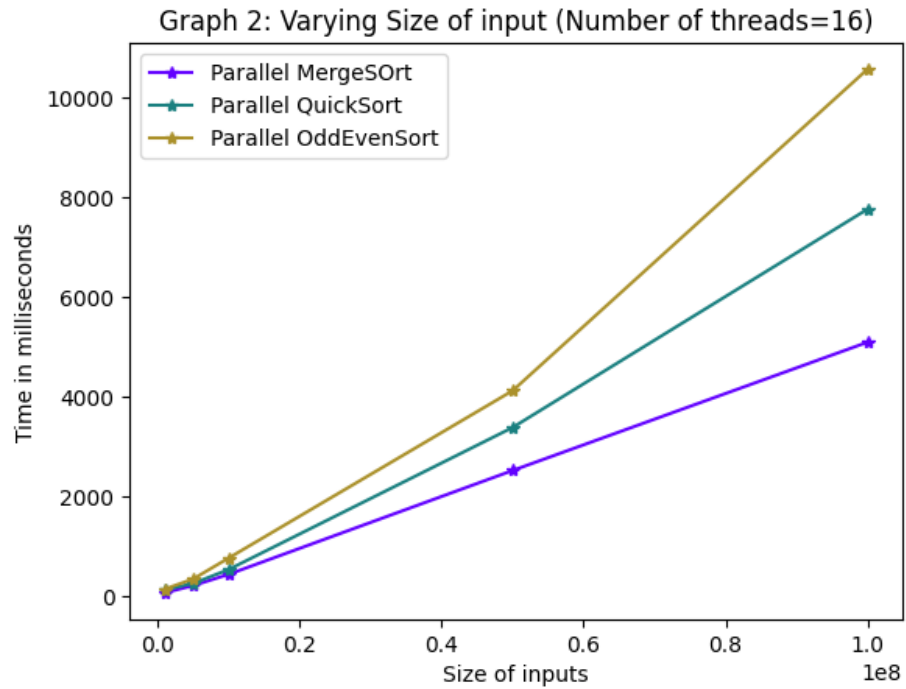
We performed the following two experiments

- We vary the number of threads in 2^i where i is in range $[1, 5]$, keeping the N as 10000000
- We vary the N from 1000000 to 100000000, keeping the number of threads as 16

4 Plots and observations



- We observe that parallel quicksort is faster for smaller number of threads due to better cache locality, in place operations and smaller constant factors.
- As number of threads increases, parallel mergesort catches up and performs considerably better, since the input is divided always into perfectly equal halves (unlike quicksort where pivot quality determines the division of work).
- The saturation of mergesort and quicksort at 16 threads is due to the constraints of the hardware used for testing (16 logical cores).
- The exceptionally poor performance of odd even sort at 32 threads is due to thread creation-destruction and synchronisation overhead surpassing the gains due to parallelism as the work per thread diminishes



- The performance gap between parallelised mergesort, quicksort and odd even sort becomes more distinct as the workload increases.
- The massive increase in time taken from $1e7$ to $5e7$ for all the three algorithms is due to the data going from being predominantly fetched from cache to RAM.

If we take one int to be 4 bytes, then the size of data went from roughly 40 MB to 200 MB, which is much greater than the size of L3 cache. CPU cycles required to fetch the data from RAM is 2-3 times more than the L3 cache.

References

- [1] R. Cole, “Parallel merge sort,” *SIAM Journal on Computing*, vol. 17, pp. 770–785, Aug. 1988.
- [2] D. M. W. Powers, “Parallelized quicksort and radixsort with optimal speedup,” in *Proceedings of the International Conference on Parallel Computing Technologies*, 1991.
- [3] A. N. Habermann, “Parallel neighbor sort (or the glory of the induction principle),” tech. rep., Department of Computer Science, Carnegie-Mellon University, Aug. 1972. Also available as Technical Report AD-759 248, National Technical Information Service, US Department of Commerce.
- [4] G. Baudet and D. Stevenson, “Optimal sorting algorithms for parallel computers,” *IEEE Transactions on Computers*, vol. C-27, pp. 84–87, Jan. 1978.